1. What is CRUD in FastAPI

CRUD in FastAPI means the four basic operations used in a database or API.

- **C – Create** → Add new data
- **R – Read** → Get or view data
- **U – Update** → Modify existing data
- **D – Delete** → Remove data

Example in Student API:

- **Create** → Add a new student
- **Read** → View student details
- **Update** → Change student marks
- **Delete** → Remove student record

In FastAPI, CRUD operations are created using HTTP methods:

- **POST** → Create
- **GET** → Read
- **PUT / PATCH** → Update
- **DELETE** → Delete

2. Why do we use the POST method for inserting data

We use the POST method in APIs and FastAPI to insert or create new data in a database. The POST method sends data from the client to the server securely inside the request body. When a user submits information such as registration details, login data, or a new student record, the POST method is used to store that data in the database. It is mainly used for creating new resources and helps maintain proper communication between the frontend and backend in web applications.

3. Difference between PUT and PATCH

Feature	PUT	PATCH
purpose	Update complete data	Update partial data
Data updated	Entrie data	Only selected field

Missing fields	Remove missing field	Keeps remaining data unchanged
usage	Full update	Small or partial update
example	Update full student details	Update only student marks
Http methode type	update	Partial update
Used in	FastAPI APIs	FastAPI APIs

4. **What is the path parameter**
   A path parameter in FastAPI is a value passed inside the URL path to identify a specific resource or data. It helps the API access particular information such as a student ID, product ID, or user ID. Path parameters are written inside curly braces `{}` in the URL. For example, in `/students/{id}`, the `id` is a path parameter used to fetch details of a specific student. It makes APIs dynamic and easy to use for different records.

5. **What is the request body in FastAPI**
   A request body in FastAPI is the data sent by the client to the server while making an API request. It is mainly used with methods like POST, PUT, and PATCH to send information such as user details, student records, login data, or product information. FastAPI automatically reads the request body and converts it into Python objects using models like Pydantic. For example, when adding a new student, the student's name, age, and marks are sent inside the request body.
6. **Why do we use Pydantic models**
   We use Pydantic models in FastAPI for data validation and data handling. Pydantic models help ensure that the data received from the user is correct and follows the required format. They automatically check data types such as string, integer, or email and generate errors if invalid data is entered. Pydantic also converts request data into Python objects, making API

development easier, cleaner, and more secure. It is commonly used to define request bodies and response structures in FastAPI applications.

7. **What status code is returned for successful deletion?**

In FastAPI, the status code commonly returned for a successful deletion is **200 OK** or **204 No Content**.

- **200 OK** → Returned when the server sends a success message after deleting data.
- **204 No Content** → Returned when the data is deleted successfully and no response body is sent back.

The 204 status code is mostly preferred for DELETE operations in REST APIs.

8. **How can we check whether a student exists before updating?**
   In FastAPI, we can check whether a student exists before updating by searching the database or list using the student ID. If the student record is found, the update operation is performed. If the record is not found, the API returns an error message such as "Student not found" with a status code like 404. This check helps prevent updating invalid or non-existing records and improves data accuracy in the application.

9. **What happens if duplicate IDs are inserted?**
   If duplicate IDs are inserted in FastAPI or a database, it can create data conflicts and confusion because each ID should be unique. Most databases use the ID as a primary key, so inserting a duplicate ID usually raises an error and the new data is not stored. This helps maintain data integrity and prevents multiple records from having the same identity. In FastAPI applications, developers often check whether an ID already exists before inserting new data to avoid duplication error

10. **How does FastAPI automatically generate Swagger UI?**
    FastAPI automatically generates Swagger UI using the OpenAPI standard. When developers create API endpoints in FastAPI, the framework automatically reads the routes, request methods, parameters, and Pydantic models, then creates interactive API documentation. Swagger UI allows users to view, test, and understand the API directly from the browser without writing extra code. By default, Swagger UI is available at the `/docs` URL in a FastAPI application.